

中山大学

《软件工程》期末大作业

需求分析报告

项目名称：逸仙活动云——中山大学校园活动云平台

小组成员：钟梓俊 邱剑波 张宸睿

指导老师：王可泽

2026 年 7 月 16 日

目录

1	项目概述	1
1.1	项目背景	1
1.2	项目目标	1
1.3	项目范围	1
2	用户需求分析	2
2.1	用户痛点	2
2.2	核心用户需求	2
2.3	用户角色	2
2.4	用户使用场景	3
3	功能需求分析	5
3.1	功能模块划分	5
3.2	核心功能需求	5
3.3	功能优先级	7
3.4	功能需求用例图演示	7
4	非功能需求	7
4.1	性能需求	7
4.2	易用性需求	8
4.3	可维护性需求	8
5	数据需求	8
5.1	数据类型与结构	8
5.2	数据存储	9
5.3	数据量估算	9
5.4	数据安全与备份	9
6	可行性分析	10
6.1	技术可行性	10
6.2	设备可行性	10
6.3	进度可行性	11

1 项目概述

1.1 项目背景

校园活动是高校学生学习生活的重要组成部分。然而，活动信息分散于学院网站、学生组织公众号、通知群、线下海报等多个渠道，发布时间不统一、格式不一致，学生需要反复在不同平台间查找。活动临近时，报名入口遗失、时间冲突、详情不完整等问题也时常出现。对活动组织者而言，活动发布、审核、报名统计和通知推送缺少统一平台支撑，重复劳动多，管理经验难以沉淀。

本项目“逸仙活动云”面向校园活动信息的发现、发布、审核、参与和治理场景，建设一个 Web 端校园活动信息平台。系统前端采用 Vue 3、TypeScript、Vite、Pinia、Vue Router 和 Element Plus，后端采用 Flask、SQLAlchemy、JWT、蓝图与服务层结构，并提供 SQLite 本地运行路径以及 PostgreSQL、Redis、Celery 的容器化部署入口。

1.2 项目目标

项目目标概括为“让活动被看见、被管理、被可信地参与”。具体包括：

1. 为学生提供统一的活动浏览、检索、报名、收藏、日历和订阅入口，降低发现活动和管理个人安排的成本。
2. 为活动发布者提供活动草稿、提交审核、审核反馈查看和活动维护能力，减少多渠道重复发布。
3. 为管理员提供发布者申请审批、活动审核、数据源管理、抓取日志和审计日志查看能力，保证平台内容可信、操作可追溯。

1.3 项目范围

项目范围界定如下：

1. **功能范围：**系统覆盖活动发现与检索、身份认证与角色授权、活动发布与审核、个人参与管理（报名、收藏、日历）、发布者申请管理、知识关联与订阅通知、数据源与内容质量管理、审计与安全治理八项功能域（详见第 3 节）。
2. **系统范围：**交付物包括 Vue 3 前端单页应用、Flask REST API 后端、本地 Mock 联调服务以及 Docker Compose 容器化部署拓扑。
3. **约束条件：**采用关系型数据库保证数据一致性；JWT + RBAC 实现服务端授权，前端不以按钮隐藏替代后端权限校验；外部邮件、搜索和 AI 服务为可选依赖，不可用时核心浏览、认证与审核流程须能降级运行；前端支持 360px 以上视口宽度。
4. **排除项：**移动原生应用、校级统一身份认证、在线支付、生产级监控告警平台、多语言国际化等不属于本次项目范围。

2 用户需求分析

2.1 用户痛点

1. 信息分散：活动分布在多个渠道，用户无法通过一个入口完整了解校园活动。
2. 检索效率低：缺少统一分类、关键词搜索、状态筛选和排序能力，用户难以及时找到目标活动。
3. 参与管理弱：报名、收藏、日历和提醒分散在不同工具中，学生容易忘记活动时间或重复报名。
4. 发布成本高：组织者需要在不同平台重复编辑和发布活动，审核结果反馈也不集中。
5. 内容可信度不足：来源、审核状态、重复内容和质量问题难以追踪，管理员缺少治理工具。
6. 协作和演化困难：若缺少统一需求基线、接口约束和测试证据，后续迭代容易出现前后端字段不一致、权限遗漏和回归缺陷。

2.2 核心用户需求

核心用户需求以“用户价值”为中心描述：

1. 学生需要快速发现可信、完整、可筛选的校园活动信息。
2. 学生需要把报名、收藏、日历和通知统一管理，避免遗漏重要活动。
3. 发布者需要低成本提交活动，并清楚知道审核进度和处理结果。
4. 管理员需要以角色权限控制平台内容质量，并能够追溯关键操作。
5. 开发和运维人员需要通过清晰配置、自动化测试和部署文档保证项目可交付、可复现、可继续演化。

2.3 用户角色

根据调研，拟确定平台包含以下五种用户角色：

角色	角色说明	主要需求
访客	未登录用户，主要用于浏览公开活动。	查看活动列表、搜索活动、查看详情，并在需要报名或收藏时被引导登录。
普通用户	已注册登录的学生用户，系统默认角色为 <code>viewer</code> 。	管理个人资料，报名/取消报名，收藏活动，加入个人日历，订阅感兴趣内容，接收通知。
发布者	经管理员审批后获得发布权限的用户，角色为 <code>publisher</code> 。	创建活动草稿，编辑活动内容，提交审核，查看审核状态和驳回原因。
管理员	负责平台内容治理和权限管理，角色为 <code>admin</code> 。	审核活动，审批发布者申请，维护数据源，查看抓取日志与审计日志，处理异常内容。
运维/开发人员	负责部署、配置、测试和故障排查。	使用环境变量配置系统，运行自动化测试，查看日志和健康检查结果，完成版本发布与回滚。

2.4 用户使用场景

下表展示不同用户角色如何通过我们的平台解决上述存在的痛点：

场景	触发条件	期望结果
活动发现	学生希望了解近期讲座、比赛或社团活动。	用户可在首页或活动列表中按关键词、分类、状态和排序方式筛选，并进入详情页查看时间、地点、主办方和正文。
活动参与	用户决定参加某个活动。	登录后完成报名，可取消报名，可收藏活动，并可将活动加入个人日历或导出 ICS。
内容发布	发布者需要发布一项新活动。	发布者先保存草稿，补全必要信息后提交审核，管理员审核通过后活动对外展示。

场景	触发条件	期望结果
审核治理	管理员发现待审核活动或发布者申请。	管理员查看详情，选择通过或驳回并填写原因，系统记录审计日志。
订阅通知	用户关注某类知识节点或关键词。	当新活动匹配订阅条件时生成站内通知，用户可查看并标记已读。
异常处理	网络失败、未登录、越权或数据为空。	前端展示明确的加载、空态、错误提示或登录引导，后端返回稳定 JSON 错误信息。

3 功能需求分析

3.1 功能模块划分

模块	主要功能	面向角色
活动发现模块	首页活动推荐、活动列表、关键词搜索、分类/状态筛选、活动详情。	访客、普通用户
身份认证模块	图形验证码登录、邮箱验证码注册、JWT 会话、用户资料、角色鉴权。	访客、普通用户、发布者、管理员
个人参与模块	报名、取消报名、收藏、取消收藏、个人日历、ICS 导出。	普通用户
活动发布模块	草稿创建、活动编辑、提交审核、查看审核结果。	发布者、管理员
审核管理模块	活动审核、批量审核、发布者申请审批、审核意见记录。	管理员
知识与通知模块	知识节点关联、订阅管理、通知生成、已读标记。	普通用户、发布者、管理员
数据治理模块	数据源维护、抓取日志、内容质量分、疑似重复标识、审计日志。	管理员、运维人员
异常与通用模块	加载态、空态、错误态、401/403 处理、接口统一错误响应。	全部角色

3.2 核心功能需求

编号	功能需求	需求描述与验收准则
FR-01	活动浏览与检索	访客和登录用户可查看已发布活动列表；支持关键词、分类、状态、排序和分页；详情页展示标题、时间、地点、主办方、正文、附件或来源等信息。

编号	功能需求	需求描述与验收准则
FR-02	身份认证与角色授权	系统提供验证码登录、邮箱验证码注册和 JWT 会话；未登录访问受限资源返回 401，角色不匹配返回 403；前端路由和后端接口均执行权限控制。
FR-03	活动发布与审核	发布者可创建和编辑草稿，提交后进入 <code>pending_review</code> ；管理员可通过或驳回，状态至少包括 <code>draft</code> 、 <code>pending_review</code> 、 <code>published</code> 、 <code>rejected</code> ；审核意见可回显。
FR-04	个人参与管理	登录用户可报名、取消报名、收藏、取消收藏、加入/移出个人日历，并导出 ICS；同一用户对同一活动的报名、收藏和日历关系保持幂等。
FR-05	发布者申请管理	普通用户可提交发布者申请；管理员查看申请材料并审批；审批通过后用户角色变更为 <code>publisher</code> 。
FR-06	知识关联与订阅通知	活动可关联知识节点或关键词；用户可订阅节点/关键词；系统在匹配活动产生时生成站内通知，并支持通知查看和已读标记。
FR-07	数据源与内容质量管理	管理员可维护数据源，查看抓取日志；活动可记录来源、质量分和疑似重复标识；外部 URL 抓取需校验协议、域名和内网地址，降低 SSRF 风险。
FR-08	审计与安全治理	管理员可查看关键操作审计记录；密码只保存哈希，Token 不进入 URL，敏感日志脱敏；管理类导出和高权限接口仅限管理员访问。

3.3 功能优先级

优先级	功能范围	说明
Must	FR-01、FR-02、FR-03、FR-04、FR-05、FR-08	支撑项目可演示和核心业务闭环，是期末提交必须覆盖的范围。
Should	FR-06、FR-07、统一异常处理、接口契约、基础测试证据	强化平台差异化、可信治理和工程质量，应尽量完成并提供证据。
Could	外部搜索、AI 增强推荐、更多统计看板、更完整的无障碍审计	对体验和扩展有帮助，但依赖外部服务或额外工期，可作为演化方向。
Won't 本阶段	移动原生 App、支付、校级统一身份认证、正式生产监控平台	超出课程项目当前边界，不作为本次验收必需功能。

3.4 功能需求用例图演示

详细图例请见“系统建模报告”。

4 非功能需求

4.1 性能需求

- API 响应时间：**读接口（活动列表、详情、搜索）在本地/CI 环境下 P95 延迟不超过 500ms；写接口（登录、注册、报名、审核）P95 延迟不超过 1s（含邮件验证码发送）。
- 前端加载时间：**首页首屏加载时间在模拟校园网环境下不超过 2s；活动列表页采用分页（默认每页 20 条，最大不超过 100 条）和骨架屏加载态，避免用户等待时无反馈。
- 并发能力：**系统应支持至少 500 用户同时在线（预发布环境参考值），核心读接口在 50 并发请求下错误率不超过 1%。
- 分页与数据量：**所有列表接口必须分页返回，单页数量由服务端限制，响应中包含 items、page、per_page、total 等分页元信息。
- 超时与降级：**搜索接口超时设置为 10s，AI 增强和外部抓取等耗时能力超时设置为 30s；超时或服务不可达时以降级结果或明确错误提示返回，不得阻塞活动浏览、登录、报名和审核等核心路径。

4.2 易用性需求

1. 访客应能在不登录的情况下完成活动浏览、筛选和详情查看；登录只在报名、收藏、日历、发布和管理等受限操作前触发。
2. 主要页面在桌面和窄屏（支持最小视口宽度 360px）下保持可读，不应出现横向溢出、按钮文字遮挡或关键信息重叠。
3. 对加载中、无数据、请求失败、未登录、越权和提交成功等状态提供明确反馈。
4. 活动详情页应突出时间、地点、主办方、报名入口和状态，降低用户判断成本。
5. 表单校验应靠近输入位置提示原因，提交失败时保留用户已填写内容。

4.3 可维护性需求

1. 前端按 `views`、`components`、`api`、`stores`、`composables` 分层，避免页面直接散落接口细节。
2. 后端采用应用工厂、蓝图、服务层和模型层结构，领域规则尽量沉淀在服务层，便于测试和复用。
3. 需求编号、接口、页面、测试用例和缺陷记录应保持可追踪，例如 `FR-03` 对应活动发布页、提交审核接口、审核接口和发布审核测试用例。
4. 后端服务层函数和前端业务工具函数应保持不低于 70% 的测试覆盖率；核心服务模块（审核、权限、质量）覆盖率目标不低于 85%。
5. 配置项通过 `.env.example` 和环境变量管理，密钥、数据库连接、外部服务地址不硬编码进入源码。
6. 需求变更应经过影响分析，明确受影响的页面、接口、数据结构、测试和部署配置；紧急修复发布后也应补充回归用例。

5 数据需求

5.1 数据类型与结构

系统涉及的核心数据类型及其关系如下：

1. **用户数据**：用户账号（用户名、邮箱、密码哈希）、角色（`viewer` / `publisher` / `admin`）、个人资料、发布者申请记录。
2. **活动数据**：活动基本信息（标题、摘要、正文、时间、地点、主办方）、状态（`draft` / `pending_review` / `published` / `rejected`）、来源、质量分和审核意见。

3. **关系数据**：用户与活动之间的报名关系、收藏关系、日历事件关系，均由 (`user_id`, `activity_id`) 唯一约束保证幂等。
4. **知识数据**：知识节点（名称、类型、别名）、活动与节点的关联关系、用户订阅记录。
5. **审计与日志数据**：管理操作审计日志（操作者、动作、目标类型、目标 ID、时间）、数据源抓取日志、搜索日志。
6. **通知数据**：站内通知（接收人、内容、关联活动、已读状态、生成时间）。

5.2 数据存储

1. 系统采用关系型数据库（PostgreSQL）作为主存储，开发环境可使用 SQLite 替代，两者通过 SQLAlchemy 抽象层切换。
2. 数据库表结构通过 SQLAlchemy 模型定义，当前环境使用自动建表，生产环境须使用受控迁移工具（如 Alembic）。
3. 缓存与队列数据（会话、限流、抓取任务）存储在 Redis 中，可独立部署。
4. 容器化部署时，PostgreSQL 和 Redis 数据挂载为 Docker 卷，确保容器重启后数据不丢失。

5.3 数据量估算

1. **用户规模**：以中山大学在校生约 5 万人为参考，预计活跃用户约 5000–20000 人。
2. **活动规模**：每学期约 200–500 场活动，全年约 500–2000 条活动记录。
3. **关系数据**：报名、收藏、日历事件等关系数据预计约 5–20 万条。
4. **日志规模**：审计日志和搜索日志随使用量线性增长，单条日志不超过 1KB，年增长预计不超过 200MB。

上述数据量在 PostgreSQL 和 SQLite 的承载范围内，无需专门的分布式存储方案。

5.4 数据安全与备份

1. **敏感数据保护**：密码使用哈希（如 Werkzeug 的 `generate_password_hash`）存储，不保存明文；JWT 不进入 URL 参数；日志中对用户邮箱、验证码等敏感信息脱敏处理。
2. **唯一约束与幂等**：报名、收藏、日历关系通过数据库唯一约束提供最终一致性和幂等保证；业务层在写入前同时进行校验。
3. **备份策略**：开发环境数据量小，可手动备份 SQLite 文件；部署环境建议对 PostgreSQL 执行每日全量备份，定期验证恢复流程。

4. **数据隔离**：不同环境（开发/测试/预发布/生产）使用独立数据库，互不干扰。

6 可行性分析

6.1 技术可行性

系统采用的技术栈均为当前业界成熟方案：

1. **前端**：Vue 3 + TypeScript + Vite + Pinia 是主流前端开发组合，Vue 3 自 2020 年正式发布以来已拥有完整生态和社区支持；TypeScript 的静态类型系统可在编译阶段捕获接口字段不匹配等问题；Vite 提供秒级热更新和高效构建；Pinia 作为 Vue 官方推荐状态管理库，API 简洁且 TypeScript 支持完善。团队通过课程实践已掌握上述技术。
2. **后端**：Flask 作为轻量级 Python Web 框架，搭配 SQLAlchemy ORM 和 JWT 扩展，能快速实现 REST API；蓝图（Blueprint）机制支持模块化路由组织，服务层模式便于业务逻辑测试。Flask 生态成熟，相关文档和社区资源丰富。
3. **容器化与部署**：Docker + Compose 是容器编排的标准方案，团队已编写完整的 Dockerfile 和编排配置，可在 Linux 服务器上快速重建运行环境。
4. **持续集成**：GitHub Actions 作为 CI 平台，已配置前端构建、单元测试和端到端测试工作流，确保每次提交的可验证性。
5. **外部依赖降级**：邮件验证码、外部搜索和 AI 服务均设计为可选能力，当服务不可达时系统以降级模式运行，不阻塞核心业务路径。

综上，系统所依赖的各项技术均已成熟且在课程项目中得到验证，不存在重大技术风险。

6.2 设备可行性

1. **开发设备**：项目在普通 Windows 笔记本上完成全部开发和测试，无需专用开发硬件。
2. **运行环境**：开发阶段使用 SQLite 数据库和 Mock 服务，无额外基础设施依赖；部署阶段需要一台支持 Docker 的 Linux 服务器或云主机，用于运行 PostgreSQL、Redis 和 Celery 等容器化服务。
3. **CI 环境**：GitHub Actions 提供免费的 Ubuntu 运行环境，已配置浏览器（Chromium）和 Node.js 22 环境，满足前端自动化测试需求。

团队已具备全部所需的开发和运行设备，无需新增采购。

6.3 进度可行性

1. **开发周期：**课程项目周期约为 12 周，前端与后端采用并行开发模式，通过预定义的接口契约和 Mock 服务解耦联调依赖，降低串行等待时间。
2. **优先级管理：**采用 MoSCoW 方法划分需求优先级，Must 功能（FR-01 至 FR-05、FR-08）构成核心业务闭环，优先交付；Should 和 Could 功能按剩余时间弹性安排。
3. **质量保障：**CI 流水线自动执行构建、类型检查和测试，减少手工回归成本；端到端测试覆盖关键用户旅程，避免核心功能在迭代中退化。
4. **主要风险与应对：**前后端接口字段对齐需通过契约矩阵和联调会议保证；外部服务（邮件、AI）集成若超时可推迟至后续迭代，不影响核心演示。

在 MoSCoW 优先级管理和并行开发策略保障下，团队能够在课程周期内交付满足验收标准的产品。